PC-2025/26 SLIC superpixel

Albi Sadikaj
albi.sadikaj@edu.unifi.it

Abstract

In this project, I present an implementation and optimization of the well-known image segmentation algorithm, SLIC (Simple Linear Iterative Clustering), based on k-means and the CIE-Lab color space. The objective is to quantitatively evaluate the performance of both sequential and parallel versions, analyzing scalability and architectural bottlenecks. The parallelization is based on OpenMP and is used to parallelize the assignment and centroid update phases, while checking that no race conditions occur in the non tile-based version. A lock-free tile-based version was also designed and evaluated, revealing that reduced synchronization overhead does not necessarily translate to better performance.

1. Introduction

The objective of this project is the development, optimization and performance analysis of both a sequential and parallel version of the [1]SLIC algorithm (Simple Linear Iterative Clustering). SLIC is a segmentation technique that groups the pixels of an image in superpixels, local regions of the image that maintain the borders of the objects intact and sharing similar color. Superpixels are a fundamental part of the Computer Vision pipeline because they reduce the complexity of the images for other algorithms.

1.1. Motivation and computational challenges

What SLIC does is an adaptation of the k-means clustering algorithm in a five dimension space, combining both position (x, y) and CIE-Lab color space (L, a, b). Despite the inherently "data-parallel" nature of the algorithm, the actual efficient implementation has the following challenges:

- **Memory access**: SLIC is typically a memory-bandwidth bound algorithm, making the management of data access patterns critical.
- **Data locality**: The phase of assigning pixels to superpixel centers requires local scans that can generate a high number of cache misses if not properly managed.
- **Synchronization**: The center update phase requires the aggregation of statistics from thousands of pixels. The use of an efficient synchronization mechanism allow the avoidance of race conditions without penalizing scalability

1.2. Project Objectives

The main objective of this project is to quantitatively compare a sequential baseline with multiple parallelization strategies based on the OpenMP standard. The main focuses are:

- **Memory layout**: Evaluating the impact of AoS (Array of Structures) versus SoA (Structure of Arrays) data structures on cache locality and overall performance
- **Cache optimization**: Implementing a tile-based processing strategy to improve data reuse and avoid race conditions
- **Scalability analysis**: Evaluating the speedup and efficiency across different thread counts and tile sizes.

2. Algorithm description

The SLIC algorithm generates superpixels by clustering pixels based on their color similarity and spatial proximity in the image plane. The algorithm operates in the five-dimensional CIE-Lab+XY space, where $[L, a, b]$ represents the colors and $[x, y]$ represents the pixel coordinates.

2.1. Sequential algorithm overview

The sequential execution can be divided into five distinct steps:

1. **Cluster initialization**: The algorithm begins by uniformly sampling K initial cluster centers on a regular grid across the images. The grid interval is defined as $S = \sqrt{N/K}$, where N is the total number of pixels.
2. **Center perturbation**: To avoid initializing a center on an edge pixel or a noisy region, disrupting the clustering process, each center is moved to a position corresponding to the lowest gradient magnitude within a localized 3×3 neighborhood.

3. **Assignment step (iterative loop)**: For each cluster center, the algorithm searches for the best matching pixels within a bounded $2S \times 2S$ window centered on the cluster. The distance is computed as a weighted combination of the color distance (in the CIE-Lab space) and the spatial distance using the following formula:

$$\begin{aligned} d_{lab} &= \sqrt{(l_k - l_i)^2 + (a_k - a_i)^2 + (b_k - b_i)^2} \\ d_{xy} &= \sqrt{(x_k - x_i)^2 + (y_k - y_i)^2} \\ D_s &= d_{lab} + \frac{m}{S} d_{xy} \end{aligned} \tag{1}$$

   where m is a value between 1 and 20 that indicates how much pixel position influences the final cluster. If the computed distance is smaller than the pixel's currently stored minimum distance, the pixel's label and distance are updated. Since $D_s \geq 0$, squaring preserves the ordering of distances, as x^2 is strictly monotonically increasing for $x \geq 0$. This makes the comparison equivalent while avoiding two square root computations per pixel.

4. **Update step and error calculation (iterative loop)**: Once all pixels have been assigned, the new spatial and colorimetric coordinates for each cluster center are recalculated as the mean of all pixels carrying that center's label. The residual error (L_1 distance between the old and new centers) is computed to check for convergence. This step and the previous are generally repeated 10 times, but it can be changed to use the error or any other amount.

5. **Enforce connectivity**: The k-means clustering has a problem, it does not guarantee spatial contiguity, occasionally leaving isolated patches of pixels ("islands") disconnected from their main superpixel body. To enforce connectivity, the algorithm identifies all distinct contiguous segments belonging to each label using a Breadth-First Search (BFS) approach. For any label containing multiple disjoint segments, the largest segment is retained as the primary superpixel, while the smaller isolated islands are reassigned to the neighboring cluster with which they share the longest boundary.

2.2. Parallelizable sections

Profiling the SLIC algorithm reveals that most of the computational workload is in the iterative loop. The strategy followed for each step is the following:

- **Steps 1 & 2 (initialization and perturbation)**: The least impactful parts of the algorithm, trivially parallelized without problems. The only thing to avoid is forgetting the critical section in the initialization.

- **Step 3 (assignment)**: While this is the most computationally intensive step, it is also easy to parallelize, since all that is needed is a simple `#pragma omp parallel for` on the main loop to do it. The only issue that arises is the handling of different threads accessing the same pixels at the same time. This can be fixed with the use of either `reductions` or `std::mutex`. I opted for the second option since it allows for a simpler solution, less impact on memory usage and generally faster execution.

- **Step 4 (update and error calculation)**: This step is much less computationally intensive than the previous one while also being just as easy to parallelize. Just as the previous step, there can be race conditions on this step as well on both the calculation for the new cluster center and the error. To solve both of these I used a reduction clause.

- **Step 5 (enforcing connectivity)**: Traditionally, the connected-components algorithm is implemented sequentially due to its unpredictable memory access patterns and region-growing nature. However, the connectivity problem can be reformulated to expose multiple levels of parallelism. First, the gathering of pixel coordinates per cluster can be parallelized using thread-local buffers. Second, the BFS traversal to identify isolated islands can be executed independently for each cluster. Finally, the reassignment of orphan pixels to neighboring clusters can be performed concurrently across different labels. Because the workload per cluster varies significantly based on the presence of fragmented islands, OpenMP's `schedule(dynamic)` is perfectly suited to maintain load balancing during this phase.

3. Parallel Implementation

The parallelization strategy was implemented in C++ using the OpenMP API. The primary objective was to maximize thread utilization while mitigating the memory bandwidth bottleneck inherent to the SLIC algorithm.

3.1. Data structures and memory layout (SoA vs AoS)

A critical architectural decision was abandoning the traditional Array of Structures (AoS) representation for pixels (e.g. an array of `struct {float L, a, b; int x, y}`) in favor of a **Structure of Arrays (SoA)** layout. This layout is highly beneficial for the assignment and update steps. When calculating distances or updating centroids, the CPU can sequentially stream the required color channels and coordinates into the cache, maximizing cache-

line utilization significantly reducing cache misses compared to the scattered memory accesses of an AoS layout.

3.1.1 SoA impact analysis

Contrary to initial expectations, adopting a SoA layout for the image buffer did not yield significant speedups compared to the AoS layout (cv::Mat). This is because the assignment phase accesses image pixels strictly sequentially, allowing the CPU's hardware prefetcher to perfectly hide memory latencies even with AoS. Furthermore, SoA forces the CPU to maintain three separate memory streams (for L, a, and b), increasing TLB pressure. While adopting a SoA layout for the image did not impact performance, adopting it for the cluster centers unexpectedly provided a consistent performance improvement. This outcome was counterintuitive due to the nature of the algorithm: distance calculations require simultaneous access to all cluster components (both color and spatial coordinates). This vertical access pattern is traditionally better suited for an AoS layout to minimize cache misses. However, this theoretical drawback was outweighed by the architectural advantage SoA provided: it unlocked the ability to use native OpenMP array reductions, completely eliminating the computational overhead of explicit critical sections.

3.2. Parallelization strategy

3.2.1 The assignment phase (memory access and mutexes)

The `parBestMatchPixelNeighborhood` function is the core bottleneck. The outer loop iterates over the cluster centers using `#pragma omp parallel for schedule(dynamic)`. dynamic scheduling was chosen because cluster centers located near image boundaries naturally process fewer pixels than those in the center, leading to workload imbalance with static scheduling. Inside the loop, threads scan the local 2S×2S neighborhood. To prevent race conditions when two adjacent clusters attempt to update the minimum distance of the same pixel simultaneously, an array of `std::mutex` is utilized (`row_mutexes`). Locking is performed at the row level rather than per-pixel to drastically reduce the synchronization overhead, striking a balance between thread safety and lock contention.

3.2.2 The update phase (Reduction)

In `parNewClustersCenters`, accumulating the coordinates and color channels of thousands of pixels into the new cluster centroids presents a massive reduction problem. To solve this efficiently, the SoA layout of the clusters proves crucial. Because the color channels and coordinates are stored as primitive continuous arrays, OpenMP's native array reduction mechanism can be elegantly employed: `#pragma omp parallel for schedule(static)reduction(+: ptr\_l[:real\_k], ptr\_a[:real\_k], ...)` This directive automatically creates and manages thread-local copies of the accumulation arrays. Each thread accumulates the data for its assigned chunk of the image independently and completely lock-free. At the end of the parallel region, OpenMP implicitly merges the thread-local arrays into the global centroid arrays, bypassing the need for severe performance bottlenecks like explicit `#pragma omp critical` sections or the manual management of thread-local buffers.

3.2.3 Parallel connectivity

As identified in 2.2, enforcing connectivity was fully parallelized in `parEnforceConnectivity`. The implementation relies on three parallel stages:

1. **Thread-local gathering**: Pixels are grouped by their cluster IDs using thread-local vectors to avoid allocation bottlenecks

2. **Concurrent BFS executions**: While the standard Breadth-First Search (BFS) algorithm is inherently sequential due to its queue-based nature, the island discovery process exposes coarse-grained parallelism. Instead of attempting to parallelize a single BFS traversal, the algorithm executes multiple independent BFS instances concurrently. Using `schedule(dynamic)`, threads process different cluster IDs simultaneously, safely running local sequential `floodFillBFS` operations without interfering with one another.

3.3. OpenMP implementation details

The parallel version makes use of different OpenMP directives that allow multiple threads to work while avoiding interference between each other. The Directives used can be divided into the following four groups:

- Work-sharing and scheduling: Workload distribution is managed through `#pragma omp parallel for` directive, with scheduling policies explicitly defined to maximize thread utilization:

  - **schedule(static)**: applied during the cluster perturbation and update phases. In these steps, the computational cost per cluster or per pixel is constant and predictable. Static scheduling partitions the iterations into equal chunks at compile-time, minimizing runtime overhead.
  - **schedule(dynamic)**: strictly enforced in the *Assignment* phase and the *Connectivity* phase. In

the Assignment step, clusters located near the image boundaries process fewer valid pixels than those in the center. Similarly, during the Connectivity step, the size of disconnected islands discovered by the BFS varies unpredictably. Dynamic scheduling effectively mitigates load imbalance by assigning new chunks to threads dynamically at runtime as soon as they finish their current work, ensuring no thread sits idle.

- Synchronization and Barriers : To minimize thread contention, explicit synchronization is kept to an absolute minimum:
  - **nowait clause**: used within the `#pragma omp for nowait` directives during the *Initialization* and *Connectivity* (pixel gathering) phases. By removing the implicit synchronization barrier at the end of the work-sharing loop, threads that finish populating their private local buffers can immediately proceed to the merging stage without waiting for slower threads.
  - **critical sections**: utilized exclusively at the end of parallel regions to safely merge thread-local structures (e.g., `private\_centers` or `local\_pixels`) into global `std::vector` containers. This design isolates the locking overhead to a single quick operation per thread, rather than locking on individual pixel insertions.
- **Lock-Free Aggregation**: as detailed in the Update phase analysis, `#pragma omp parallel for reduction(+: ...)` is employed to implicitly aggregate the new cluster coordinates. By passing the primitive arrays extracted from the SoA layout (e.g., `ptr\_l[:real\_k]`), OpenMP automatically manages thread-local accumulators and performs a lock-free global reduction at the end of the loop.

While reduction is ideal for the update phase, it is not always the best option. This is noticeable in the *assignment step*, which features the use of `C++ std::mutex` instead of a `reduction clause`. While the reduction clause is a valid option, it would require each thread to maintain a private copy of the entire label and distance arrays, generating prohibitive memory overhead. The solution was to use a fine-grained and double-checked locking, which allows for a much smaller overhead compared to the reductions while giving the same results.

3.4. Challenges encountered and solutions: The Tiling Trade-off

One of the primary challenges during parallelization was managing the synchronization overhead in the Assignment phase. Although the use of `row_mutexes` prevented race conditions through a fine-grained approach, the continuous acquisition of locks in the innermost loop (hot loop) still represented a potential bottleneck.

To attempt the complete elimination of the need of any kind of synchronization mechanism, an alternative architecture was designed: the Tile-based approach. The core idea inverts the algorithm's logic: the image is divided into disjoint blocks (e.g., 32×32 pixels). For each tile, the thread determines which superpixel centers intersect the block's area; subsequently, it iterates over the pixels within the tile, computing the distance only to those local centers. Because threads process strictly disjoint image regions, memory writes are inherently safe, making the assignment phase 100% lock-free.

3.4.1 Analysis of the results: The failure of Tiling

Despite the theoretical advantage of a lock-free execution, benchmarks demonstrated that the Tile-based version was systematically slower than the standard parallel version with mutexes. This performance degradation was analyzed and traced back to two fundamental issues introduced by the new architecture:

1. **Increased pixel evaluations (Geometric Inefficiency)**: In the standard algorithm, each pixel is compared exactly and exclusively with the centers whose nominal search area ($2S \times 2S$) covers it. In the tile-based approach, if a superpixel's search area intersects even a single corner of the tile, all pixels within that tile will be evaluated against that center. This generates a significantly higher average number of Euclidean distance computations per pixel compared to the standard method, completely negating the time saved by removing the mutex.

2. **Setup and memory management overhead: The standard parallel version has near-zero architectural overhead**: the mutex array is initialized only once, and threads solely require input pointers to the function. Conversely, the Tiling approach forces each thread to continuously declare, clear (`.clear()`), and repopulate support vectors (`local_centers`) for every single processed tile. In a 512×512 image using 32×32 tiles, this costly memory management operation is repeated 256 times. As the image resolution scales, the accumulation of these setup micro-latencies severely degrades overall performance.

An additional property of the benchmarking setup is the ability to skip execution for tile sizes that are too large relative to the image dimensions. Specifically, if the number of tiles that fit within the image falls below the number of

available threads, the configuration is considered degenerate — the workload cannot be distributed evenly and some threads would remain idle for the entirety of the computation. In such cases, the benchmark skips that configuration entirely, which explains the missing entries marked with "-" in Tables 5 and 6.

4. Experimental Setup

4.1. Hardware specifications

The performance evaluations were conducted on a workstation equipped with:

Component	Specification
CPU	AMD Ryzen 7 7840HS
Cores	8 Physical
Threads	16 Logical
Cache	512 KB L1 (Data/Instr.), 8 MB L2, 16 MB L3
RAM	16 GB DDR5 5600 MHz

Table 1. Hardware configuration for benchmarking.

4.2. Software specification

The project was built and run on Windows 11 environment utilizing CMake 4.0.2 as the primary build system. The compilation was done with GCC 15.2.0 (g++) with the following flags:

- `-O3`: Optimization level 3, instructing the compiler to do everything it can to optimize the code so that it runs faster.
- `-ffast-math`: Flag that instructs the compiler to break certain rules to make calculations faster. Some rules that are not completely followed with it are: IEEE compliance in the order of instruction, assuming all math is finite, reciprocal approximations for division and reciprocal square root and no signed zero. This can result in minor errors in floating point calculation, but it can also provide increased performance for a really small imprecision.
- `-fopenmp`: Enables the ability to use OpenMP directives with the GCC compiler.

As for the capacity to correctly read, modify and save image file, OpenCV 4.12.0 was used through MSYS2. The parallel version (both non-tile-based and tile-based) were made using OpenMP 4.5 (`_OpenMP=201511`). A flag that wasn't used in the final tests is the `-march=native` which instructs the compiler to try and use AVX2 or even AVX512, if supported; The reason for not using is that the times were basically the same as without the flag entirely.

4.3. Tested parameters

To test the efficiency of the normal parallel version and the tile-based one over the sequential version, I varied the following parameters:

- **Images**: this contains all the images that would be used for the tests. Each image also has its own superpixel count. In the test I used the following settings (image, SP count): [2](COCO-1, 200), (COCO-2, 300), [3](1080p, 1000), (4k-test-2, 2000)
- **Thread count**: the thread count starts at 1 and doubles after running both parallel version until it reaches 16, which is double the number of physical CPU cores.
- **Tile size**: for the tile size we go from 16 up to 512, doubling the size at each step.

4.4. Dataset used

The dataset was composed of images selected to cover a wide range of resolutions, from low (640×480) to high (3840×2160), in order to evaluate the algorithm's scalability across different workload sizes. Standard benchmark datasets such as COCO were also used as their images, but other images come from other sites to obtain the higher resolution photos, since COCO had predominately low resolution ones.

5. Results and analysis

The correctness of each version was compared to the sequential result by looking for differences in the final image and expressing it as the percentage of pixels equal to the ones in the sequential. The performance was measured using the hardware and software setup detailed in Table 1. Something that was noticed during testing is the fact that superpixel amount didn't influence time results as much as thought, instead the major influence was the size of the image. This is to be expected since having a few extra superpixels results in only a few extra operations, while the image size drastically increases the amount of checks needed since the superpixels gain bigger areas with it.

5.1. Sequential baseline

To know the speedup of the parallel version and how accurate they are we need a baseline. The results of the implementations can be seen in Table 2

5.2. Parallel version

Something we notice from the start is that the parallel version is always slightly slower when the amount of threads is 1 and only 1. The gap increases as the image size

Image	Time ms
COCO-1	49.73
COCO-2	41.49
1080p	325.41
4k	900.04

Table 2. Average sequential execution times (ms) on different images.

Image	T=1	T=2	T=4	T=8	T=16
COCO-1	58.57	35.77	22.56	16.03	13.87
COCO-2	49.70	32.64	24.30	17.56	15.47
1080p	404.59	254	155.81	99.74	74.68
4k	1101.7	715.44	460.12	296.30	235.73

Table 3. Average parallel execution times (ms) on different images.

increases but it's only true for when there's one thread. The results of the implementations can be seen in Table 3

As said earlier, the times heavily depend on the size of the image. What can be also be seen here is that the speedup at each thread count increase is quite linear; while each step is not the same as the previous one, the same step for different images improves times by a similar factor. We can also see the speedup when compared to the sequential at Figure 1.

The efficiency of the standard parallel against the tile can be seen at Figure 3.

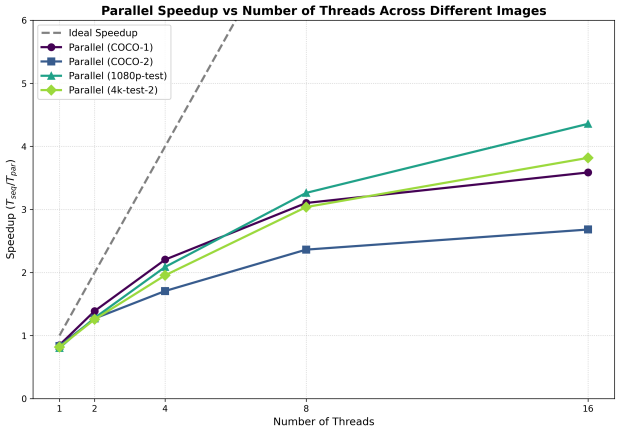

Figure 1. Parallel speedup with different thread counts on all images.

5.3. Tile-based parallel version

This implementation, while it was supposed to be the best one, it ended up being the in between. The main caveat it has is the need of a search of the best tile size based on the image used. The results in the tile-based implementation times show that a size of 16 or 32 generally gives the best results at 16 threads. It's also noticeable that, just as the non-tile-based parallel version, if the thread count is 1, the parallel is slower than the sequential; the difference in the tile-based is how much slower due to the amount of extra overhead needed to implement it.

The results with 1, 4 and 16 thread are visible at Tables 4, 5 and 6. The speedup of the tile compared to the sequential on the image 'COCO-1' can be viewed at Figure 2

	Tile size					
Image	16	32	64	128	256	512
COCO-1	72.13	79.50	101.95	165.58	345.07	933.98
COCO-2	89.86	114.27	205.09	499.41	1328.09	4175.17
1080p	573.78	637.82	690.73	1122.53	2420.78	6583.83
4k	1851.1	1738.38	1789.14	2736.99	5610.86	15013.7

Table 4. Average tile-based parallel execution times (ms) on different images with 1 thread.

	Tile size					
Image	16	32	64	128	256	512
COCO-1	25.89	29.58	39.39	84.57	227.02	-
COCO-2	35.45	45.58	81.44	246.27	1014.45	-
1080p	203.38	217.24	247.18	449.99	941.96	4509.72
4k	707.08	643.28	688.30	1067.21	2410.25	7256.11

Table 5. Average tile-based parallel execution times (ms) on different images with 4 threads.

	Tile size					
Image	16	32	64	128	256	512
COCO-1	16.06	16.70	24.95	65.49	-	-
COCO-2	19.27	24.75	58.21	226.54	-	-
1080p	85.67	89.08	109.15	270.65	907.247	-
4k	305.26	301.01	335.16	506.13	1491.62	7095.79

Table 6. Average tile-based parallel execution times (ms) on different images with 16 threads.

5.4. Performance results and scalability

To evaluate the parallel efficiency of the proposed implementation, a series of Strong Scaling tests were conducted. In this setup, the problem size (image resolution) remains constant while the number of processing elements (OpenMP threads) is increased. The 4K resolution image (4k-test-2.png) was selected as the primary benchmark to ensure a workload sufficiently large to saturate the multi-core architecture of the Ryzen 7 7840HS.

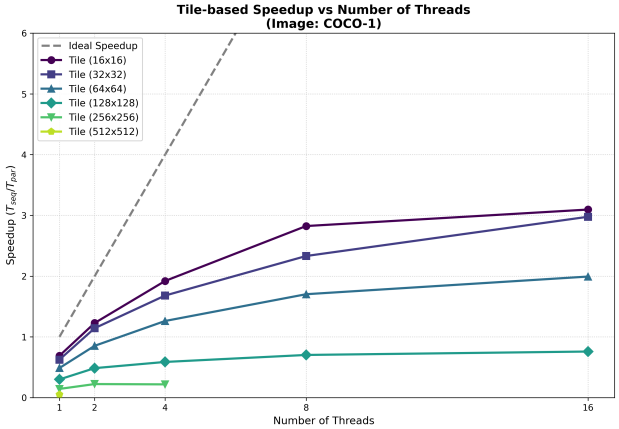

Figure 2. Speedup of tile-based parallel implementation on COCO-1 with different tile sizes.

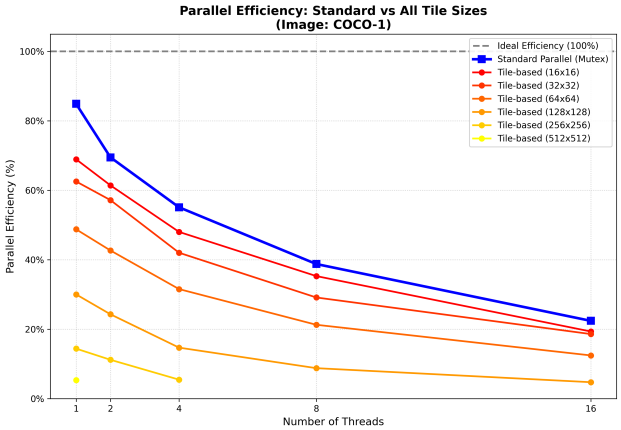

Figure 3. Efficiency of all parallel versions on image COCO-1.

5.4.1 Scaling analysis

The sequential baseline recorded an execution time of 900.04 ms. As shown in the results, the algorithm exhibits significant performance gains up to 8 threads, which corresponds to the number of physical cores available on the test machine. At this threshold, the execution time drops to 296.30 ms, yielding a speedup of 3.04x.

When increasing the thread count to 16 (utilizing Simultaneous Multithreading - SMT), the execution time further decreases to 235.73 ms (peak speedup of 3.82x). However, the efficiency curve shows a noticeable "knee" or deceleration after 8 threads. This behavior is expected in memory-intensive applications: while SMT helps hide memory latency by interleaving instructions from different threads, the competition for shared L3 cache and memory bandwidth becomes a limiting factor once all physical cores are fully utilized.

5.4.2 Impact of workload size

A key observation in this study is the correlation between image resolution and parallel efficiency. By comparing the 4K results with the low-resolution COCO-1 dataset, a clear trend emerges:

- **High Resolution (4K)**: Larger data chunks per thread allow the CPU to maintain high throughput, effectively masking the overhead of thread creation and synchronization.
- **Low Resolution (COCO)**: The overhead of managing row_mutexes and OpenMP barriers becomes dominant, as the actual computation time per thread is extremely short (in the order of few milliseconds).

This confirms that the proposed SoA-based parallel SLIC is highly effective for modern high-resolution imaging pipelines, where the computational density justifies the parallelization overhead.

5.5. Comparison of variants: standard vs. tile-based

A comparative benchmark was executed to evaluate the performance differences between the mutex-based parallelization (run_parallel) and the lock-free Tile-based approach (run_tile). Tests were conducted using tile sizes ranging from 16×16 to 512×512.

Contrary to theoretical expectations favoring lock-free designs, the standard parallel version consistently outperformed the Tile-based variant. For instance, on the '4k-test-2' image using 16 threads, the standard version achieved an execution time of 235.73 ms, while the best-performing Tile-based configuration (32×32 tiles) took 301.01 ms. This confirms the architectural hypothesis proposed in Section 3.4: the fine-grained row_mutexes incur a negligible overhead thanks to the double-checked locking pattern. In contrast, the Tile-based method suffers from severe geometric inefficiency (redundant distance calculations for pixels outside a cluster's true radius) and massive dynamic memory allocation overhead due to the continuous clearing and re-populating of the thread-local SoA buffers for each processed tile.

5.6. Bottleneck Analysis

Profiling the application at maximum thread capacity reveals that scalability is ultimately limited by two factors:

- **Memory bandwidth**: As expected, SLIC is inherently memory-bound. The Assignment step requires streaming massive amounts of data (image channels, centroids, and distance matrices) from the RAM to the CPU registers. As 8 or 16 threads request data concurrently, the DDR5 memory bus becomes saturated, acting as a hard physical ceiling for further speedup.

- **Residual sequential fractions**: The final array merging operations inside the critical blocks and the queue management during the BFS connectivity phase cannot be perfectly vectorized or distributed, introducing localized stalls in thread execution.

- **Thread management overhead**: On low-resolution workloads (e.g., COCO dataset), the actual computational time per thread is extremely short. Consequently, the overhead introduced by OpenMP thread management and row_mutexes acquisition severely limits parallel efficiency, dropping it to roughly 20% at 16 threads.

- **Geometric inefficiency in lock-free designs**: The Tile-based implementation introduced severe thread starvation. Large tile configurations fail to fully occupy all 16 threads, while the bounding-box approach forces redundant distance computations for pixels far outside the centroid's theoretical radius, completely negating the benefits of removing mutex locks.

6. Conclusions

This project successfully achieved a highly scalable parallelization of the SLIC Superpixels algorithm. Through rigorous profiling and architectural evaluation, the memory layout was transitioned entirely to a Structure of Arrays (SoA). While theoretically more prone to TLB pressure, empirical testing proved that Pure SoA optimally leverages modern CPU hardware prefetchers, outperforming hybrid AoS solutions.

References

[1] R. Achanta, A. Shaji, K. Smith, A. Lucchi, P. Fua, and S. Süsstrunk. Slic superpixel paper. *EPFL*, 2010.

[2] COCO. train 2017 dataset, 2017.

[3] P. Nong. 1080p iamge, 2019. Princess Mononoke. Can be found at artstation.com.